

Indian Institute of Technology Madras

BS Electronic Systems

---

# Adaptive Self-Calibrating OFDM Acoustic Communication

Sending files between the speaker and microphone of ordinary devices in  
the 4–12 kHz band

---

Signal Processing Project Report

**Author:** Akshay Rukade

**Roll Number:** [Roll No.]

**Project Code (if any):** Custom

**Submission Date:** [DD-MM-YYYY]

**Instructor:** [Instructor Name]

**Team:** Akshay Rukade, J Krithika and Ritish Gajanan Sonar

# Contents

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                           | <b>3</b>  |
| <b>1 Introduction</b>                                                     | <b>4</b>  |
| 1.1 Motivation . . . . .                                                  | 4         |
| 1.2 Scope of the Project . . . . .                                        | 4         |
| <b>2 Problem Statement</b>                                                | <b>5</b>  |
| <b>3 Goals and Objectives</b>                                             | <b>6</b>  |
| 3.1 Primary Goal . . . . .                                                | 6         |
| 3.2 Specific Objectives . . . . .                                         | 6         |
| <b>4 Methodology</b>                                                      | <b>7</b>  |
| 4.1 System Overview . . . . .                                             | 7         |
| 4.2 Signal Processing Techniques Used . . . . .                           | 7         |
| 4.2.1 Filtering . . . . .                                                 | 7         |
| 4.2.2 Transform-Domain Analysis . . . . .                                 | 8         |
| 4.2.3 Additional Techniques . . . . .                                     | 8         |
| 4.3 Theoretical Background . . . . .                                      | 9         |
| <b>5 Implementation Details</b>                                           | <b>10</b> |
| 5.1 Tools and Technologies . . . . .                                      | 10        |
| 5.2 System Architecture . . . . .                                         | 10        |
| 5.3 Key Algorithms / Pseudocode . . . . .                                 | 10        |
| 5.4 User Interface . . . . .                                              | 11        |
| <b>6 Problems Faced and Solutions</b>                                     | <b>14</b> |
| 6.1 Problem 1: The synchroniser could not find the chirp . . . . .        | 14        |
| 6.2 Problem 2: Pilot interpolation aliased . . . . .                      | 14        |
| 6.3 Problem 3: Adaptive tracking made the link worse . . . . .            | 14        |
| 6.4 Problem 4: Real hardware failures that looked like DSP bugs . . . . . | 14        |
| 6.5 Problem 5: A third of the band was unusable . . . . .                 | 15        |
| 6.6 Problem 6: Repetition coding had an error floor . . . . .             | 15        |
| 6.7 Problem 7: Long frames broke under clock drift . . . . .              | 15        |
| <b>7 Results and Discussion</b>                                           | <b>16</b> |
| 7.1 Experimental Setup . . . . .                                          | 16        |
| 7.2 Quantitative Results . . . . .                                        | 16        |
| 7.3 Qualitative Results . . . . .                                         | 19        |
| 7.4 Discussion . . . . .                                                  | 22        |

|           |                                           |           |
|-----------|-------------------------------------------|-----------|
| <b>8</b>  | <b>Conclusions</b>                        | <b>23</b> |
| 8.1       | Summary of Contributions . . . . .        | 23        |
| 8.2       | Limitations . . . . .                     | 23        |
| 8.3       | Future Work . . . . .                     | 23        |
| <b>9</b>  | <b>Individual Contribution</b>            | <b>24</b> |
| <b>10</b> | <b>Annexure</b>                           | <b>25</b> |
| 10.1      | Video Demonstration . . . . .             | 25        |
| 10.2      | Source Code . . . . .                     | 25        |
| 10.3      | Live Deployment (if applicable) . . . . . | 25        |
|           | <b>References</b>                         | <b>26</b> |

# Abstract

Consumer devices all have a speaker and a microphone, but no common way to exchange data through them. We built a software OFDM modem that sends files as sound in the 4–12 kHz band, using nothing but the device’s own audio hardware. The transmitter maps QPSK symbols onto a Hermitian-symmetric 256-point IFFT, so its output is already a real audio signal. It adds a cyclic prefix, clips and re-filters the waveform to lower its peak-to-average power ratio, and prepends a linear chirp. The receiver band-pass filters the recording and finds the frame with a chirp matched filter. It then estimates the channel from a reference symbol, tracks it with pilots, and equalises with an MMSE filter. A self-calibration stage measures the hardware’s frequency response first and transmits only on usable subcarriers. Payloads are protected by a rate- $\frac{1}{2}$  convolutional code with soft-decision Viterbi decoding. The receiver estimates the sampling-clock offset between devices from its pilots and corrects it. Measured AWGN BER matches coherent-QPSK theory. Without equalisation the multipath channel carries nothing (BER 0.47 at every SNR). At a 100 ppm clock offset, adaptive tracking gives  $3.8 \times 10^{-3}$  against 0.51 for one-shot calibration. Self-calibration took the real over-the-air BER from 0.31 to  $8.3 \times 10^{-3}$ . In simulation, the convolutional code delivers 2 408 bit/s of file data and every file intact from 4 dB SNR, where repetition coding needs 18 dB. [\[Add one sentence on the two-device result.\]](#) Results are published on an interactive web dashboard.

# Chapter 1

## Introduction

Acoustic data transfer is attractive precisely because it needs no pairing, no radio and no shared network: any two devices with a speaker and a microphone can talk. It is also a hard channel. A laptop speaker and microphone in a real room form a strongly frequency-selective filter with deep nulls; the two devices run from independent crystals, so their sample clocks disagree; and a speaker is peak-limited, which penalises the high crest factor of multicarrier signals. Orthogonal frequency-division multiplexing (OFDM) is the standard answer to frequency-selective channels in Wi-Fi, LTE and DSL, and this project applies it to sound.

### 1.1 Motivation

We chose this project because it exercises nearly every core DSP idea in one working system: the DFT and its symmetry properties, filtering, correlation and matched filtering, spectral estimation, and estimation under noise. Every idea can be checked against real hardware rather than only in simulation. The "adaptive, self-calibrating" framing comes from a concrete real-world problem: a one-shot channel estimate is useless between two devices whose sample clocks drift apart.

### 1.2 Scope of the Project

**In scope:**

- A complete OFDM transmitter and receiver in the 4–12 kHz audio band.
- Pilot-based channel estimation, MMSE equalisation and adaptive tracking.
- Self-calibration of the usable band from a measured hardware response.
- CRC framing, forward error correction and blind (self-describing) frames.
- Sampling-clock-offset estimation and correction.
- BER/PRR measurement in simulation and over real audio hardware, and a web dashboard presenting the results.

**Out of scope:** inaudible (ultrasonic) operation; multi-user access; full-duplex links and feedback handshakes; bit-loading and higher-order modulation; mobile apps. Echo-based distance estimation was prototyped and dropped after the mid-term review.

## Chapter 2

# Problem Statement

Transmit an arbitrary file from one device’s speaker to a microphone, and recover it exactly, using OFDM in the 4–12 kHz band. The receiver must find the frame in a continuous recording with an unknown delay. It must estimate and equalise a frequency-selective acoustic channel with nulls up to 43 dB deep, and remain correct when transmitter and receiver sample at slightly different rates (a sampling-clock offset of tens to hundreds of ppm). It must also report bit error rate (BER) and packet recovery rate (PRR) so each stage’s contribution can be measured.

**Assumptions and constraints.** Standard 48 kHz audio hardware with no special drivers (ALSA `aplay/arecord` on Linux); a quasi-static channel over one 6.7 ms OFDM symbol; a single transmitter at a time; a speaker-to-microphone distance of a few metres; excess path length under 0.46 m (the cyclic prefix); and a peak-limited transmitter.

## Chapter 3

# Goals and Objectives

### 3.1 Primary Goal

To build and measure an adaptive, self-calibrating OFDM modem that reliably transfers files through ordinary speakers and microphones.

### 3.2 Specific Objectives

- Objective 1 (Phase 1): implement OFDM end to end — bits  $\rightarrow$  IFFT + cyclic prefix  $\rightarrow$  speaker  $\rightarrow$  microphone  $\rightarrow$  synchronisation  $\rightarrow$  FFT  $\rightarrow$  bits — and validate the measured BER against coherent-QPSK theory.
- Objective 2 (Phase 2): add pilots, least-squares channel estimation and equalisation, and quantify the BER improvement over no equalisation.
- Objective 3 (Phase 3): track the channel adaptively from the pilots and show where it outperforms one-shot calibration, particularly under sampling-clock offset.
- Objective 4: measure the real hardware response and transmit only on usable subcarriers (Stage-1 self-calibration).
- Objective 5: add forward error correction and characterise long frames under clock drift.
- Objective 6: test between two separate devices.
- Objective 7: publish an interactive dashboard showing the complete signal chain and every measurement.

# Chapter 4

## Methodology

### 4.1 System Overview

Figure 4.1 shows the end-to-end pipeline. The transmitter packetises the file, encodes it, maps it onto QPSK subcarriers with pilots, and synthesises a real audio waveform with a single IFFT per symbol. The receiver synchronises to the chirp, returns to the frequency domain, estimates and tracks the channel, corrects the clock offset, equalises and decodes.

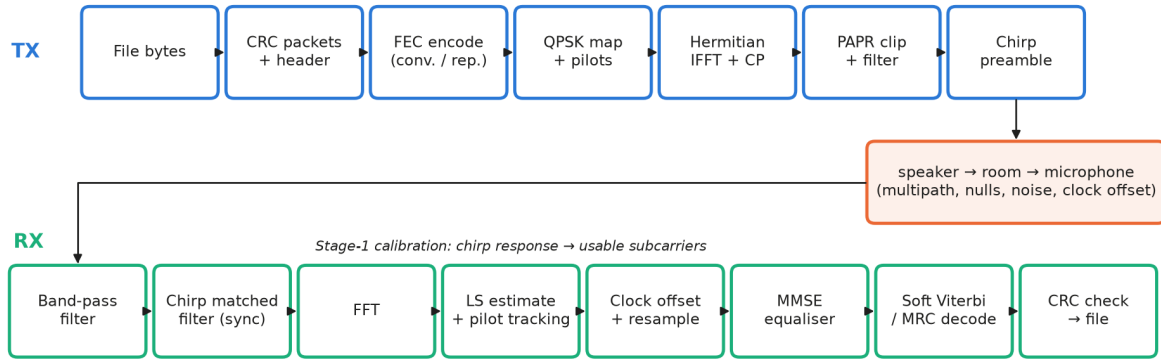


Figure 4.1: System architecture block diagram

### 4.2 Signal Processing Techniques Used

#### 4.2.1 Filtering

**Receiver front end.** On a real laptop recording, the 0–1 kHz band (fan, mains hum, desk rumble) sat 21 dB *above* the signal band. The first receiver stage is a 4th-order Butterworth band-pass IIR filter, with edges 15 % outside the 4–12 kHz band. It is applied with `filtfilt`, i.e. forward and backward. The resulting zero phase matters: an ordinary causal IIR filter would add group delay and shift the frame timing the synchroniser is about to measure.

**Peak-to-average power reduction.** After clipping at  $\gamma$  dB above RMS, the block is transformed with an FFT and every bin outside the occupied band is zeroed. This is an ideal brick-wall filter applied in the frequency domain. It removes the out-of-band splatter that clipping creates. Three clip-and-filter iterations are used.

**Equalisation** is itself a per-subcarrier filter (Section 4.2.3).

### 4.2.2 Transform-Domain Analysis

The Discrete Fourier Transform is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (4.1)$$

and OFDM uses its inverse as the modulator: each subcarrier  $k$  carries one complex QPSK symbol  $X[k]$  for one symbol period.

**Real-valued OFDM through Hermitian symmetry.** We place QPSK symbols on bins  $k \in \{22, \dots, 64\}$  of an  $N = 256$  IFFT at  $f_s = 48$  kHz ( $\Delta f = 187.5$  Hz, so 4.1–12 kHz) and set

$$X[N-k] = X^*[k], \quad X[0] = X[N/2] = 0. \quad (4.2)$$

The IDFT of a Hermitian-symmetric spectrum is purely real, so  $x[n]$  is already an audio-band signal. No complex baseband, mixer or carrier recovery is needed.

**Cyclic prefix.** Copying the last  $L_{cp} = 64$  samples to the front of each symbol makes the channel's linear convolution look circular over the FFT window, provided the channel impulse response is shorter than the prefix. Each subcarrier then sees a single complex gain:

$$Y[k] = H[k] X[k] + W[k]. \quad (4.3)$$

**Spectral measurement.** The hardware response used for self-calibration is estimated in the frequency domain from the recorded chirp,  $|H[k]| = |Y[k]|/|X_{\text{chirp}}[k]|$ , and averaged over each subcarrier's bin.

### 4.2.3 Additional Techniques

**Matched filtering and cross-correlation (synchronisation).** The received signal is correlated with the known chirp  $c[n]$ , and the envelope of the result is taken with the Hilbert transform:

$$r[m] = \left| \mathcal{H} \left\{ \sum_n y[n+m] c[n] \right\} \right|. \quad (4.4)$$

A 30 ms sweep over 10 kHz has a time–bandwidth product near 300. The correlation peak is therefore about  $1/B = 0.1$  ms wide even at 0 dB SNR, and the envelope makes timing independent of carrier phase. Detection uses the peak-to-20th-percentile ratio.

**Least-squares channel estimation** from the known reference symbol,  $\hat{H}[k] = Y[k]/X[k]$ , followed by a 3-tap moving average across frequency.

**MMSE equalisation.**

$$\hat{X}[k] = \frac{\hat{H}^*[k]}{|\hat{H}[k]|^2 + \sigma^2} Y[k]. \quad (4.5)$$

This reduces to  $1/\hat{H}$  where the channel is strong. In a 40 dB null it does not amplify the noise by 40 dB, as zero-forcing would.

**Adaptive tracking.** At the pilot subcarriers the receiver forms the correction  $C[k] = \hat{H}_{\text{pilot}}[k]/\hat{H}[k]$  and fits the physical model

$$C(k) = g e^{j(a+bk)} \quad (4.6)$$

by  $|\hat{H}|^2$ -weighted least squares, then updates  $\hat{H} \leftarrow \hat{H} (1 - \mu + \mu C)$  with  $\mu = 0.15$ . Here  $g$  is a gain change,  $a$  a common phase and  $b$  a phase slope across frequency. The slope is the signature of a timing drift, since a delay of  $\tau$  samples rotates subcarrier  $k$  by  $2\pi k\tau/N$ .

**Sampling-clock-offset estimation.** A relative clock offset  $\varepsilon$  slides the FFT window by  $\varepsilon L$  samples per symbol ( $L = N + L_{cp} = 320$ ). For each neighbouring pilot pair we form

$Z_n = P_n[i+1] P_n^*[i]$ , which cancels the common phase. Its angle grows linearly in the symbol index:

$$\angle Z_n = \phi_0 + \frac{2\pi \Delta k \varepsilon L}{N} n. \quad (4.7)$$

Unwrapping and fitting a line over the frame gives  $\varepsilon$ , with error falling as  $N_{\text{sym}}^{-3/2}$ . The receiver then resamples the recording by  $(1 + \varepsilon)$  and decodes again.

**Forward error correction.** A rate- $\frac{1}{2}$ , constraint-length-7 convolutional code with generators  $(171, 133)_8$  (free distance 10, as in IEEE 802.11a). It is decoded by the Viterbi algorithm with soft inputs, the equalised in-phase and quadrature values, so weak subcarriers carry proportionally less weight. A fixed pseudo-random interleaver breaks up the error bursts caused by spectral nulls.

**Peak-to-average power ratio (PAPR)** is  $\max |x[n]|^2 / \mathbb{E}|x[n]|^2$ . Clipping to 6 dB above RMS lowers it from 11.3 to 8.0 dB.

### 4.3 Theoretical Background

- **QPSK in AWGN.** With Gray coding, coherent QPSK has bit error probability  $P_b = Q\left(\sqrt{2E_b/N_0}\right)$ , the benchmark for Phase 1 [3].
- **Circular convolution.** A cyclic prefix at least as long as the channel impulse response turns linear convolution into circular convolution, so the DFT diagonalises the channel [1].
- **Matched filter.** Correlating with the known waveform maximises output SNR in white noise, and the resolution of a chirp is set by its bandwidth [2].
- **Clipping and filtering** for OFDM PAPR reduction [4].
- **Sampling-clock offset** in OFDM appears as a phase rotation proportional to subcarrier index and growing with time [5].
- **Viterbi decoding** is maximum-likelihood sequence detection on the code trellis [6].

## Chapter 5

# Implementation Details

### 5.1 Tools and Technologies

- **Programming Language:** Python 3
- **Libraries:** NumPy and SciPy only (all DSP written from scratch); ALSA `aplay/arecord` for audio; Matplotlib only for report figures
- **Platform:** Linux desktop (command line), plus a browser-based live demo served from the laptop; the dashboard is plain HTML/SVG with no framework
- **Hosting (if deployed):** GitHub Pages (URL in the Annexure); continuous integration with GitHub Actions (28 unit and end-to-end tests)

### 5.2 System Architecture

| Module                                       | Responsibility                                                     |
|----------------------------------------------|--------------------------------------------------------------------|
| <code>ofdm/config.py</code>                  | every system parameter, one source of truth                        |
| <code>ofdm/modem.py</code>                   | QPSK map/demap, Hermitian IFFT, cyclic prefix, pilots, chirp, PAPR |
| <code>ofdm/sync.py</code>                    | matched filter, frame detection, clock-offset estimation           |
| <code>ofdm/equalizer.py</code>               | LS estimation, adaptive tracking, MMSE equalisation                |
| <code>ofdm/calibrate.py</code>               | Stage-1 response measurement and subcarrier selection              |
| <code>ofdm/channel.py</code>                 | simulated acoustic channel (multipath, transducers, clock offset)  |
| <code>ofdm/receiver.py</code>                | scripted receive chain, band-pass front end, BER/EVM               |
| <code>ofdm/live.py</code>                    | blind receiver for self-describing frames, clock correction        |
| <code>ofdm/framing.py</code>                 | CRC packets, headers, repetition FEC, live frame format            |
| <code>ofdm/conv.py</code>                    | convolutional code, soft Viterbi decoder, interleaver              |
| <code>ofdm/audio_io.py</code>                | ALSA playback/capture, automatic gain calibration                  |
| <code>experiments/</code>                    | every measurement in this report                                   |
| <code>live_server.py</code>                  | browser live demo (type a message, hear it, decode it)             |
| <code>dashboard/</code> , <code>docs/</code> | results dashboard and its published copy                           |
| <code>tests/</code>                          | 28 unit and end-to-end tests                                       |

Table 5.1: Code modules

### 5.3 Key Algorithms / Pseudocode

**Transmitter: Hermitian IFFT with cyclic prefix** (`modem.py`):

```
def grid_to_time(grid, cfg):
```

| Parameter               | Value                   | Reason                                            |
|-------------------------|-------------------------|---------------------------------------------------|
| Sample rate             | 48 kHz                  | supported by all consumer hardware                |
| FFT size $N$            | 256                     | 6.67 ms symbol; channel static across it          |
| Cyclic prefix           | 64 samples (1.33 ms)    | covers 0.46 m of excess path                      |
| Band                    | 4–12 kHz                | speaker roll-off below, microphone roll-off above |
| Subcarriers             | 43 = 31 data + 12 pilot | pilot every 4th, both edges pinned                |
| Modulation              | QPSK, Gray coded        | 2 bits per subcarrier                             |
| Raw payload rate        | 9 300 bit/s             | 62 bits per symbol                                |
| Preamble                | 30 ms chirp, 3–13 kHz   | sync and channel sounding                         |
| FFT-window backoff      | 8 samples               | below $N/(2 \cdot 4) = 32$ aliasing limit         |
| Tracker step size $\mu$ | 0.15                    | from the step-size sweep                          |

Table 5.2: System parameters

```

spec = np.zeros((n_sym, cfg.nfft), dtype=complex)
spec[:, used] = grid # QPSK + pilots on bins 22..64
spec[:, cfg.nfft - used] = np.conj(grid) # Hermitian symmetry
x = np.fft.ifft(spec, axis=1).real * cfg.nfft
return np.concatenate([x[:, -cfg.ncp:], x], axis=1).ravel() # add CP

```

**Receiver: blind decode with clock correction (live.py):**

```

def demodulate_blind(rx, cfg):
    rxf = bandpass(rx, cfg) # zero-phase Butterworth
    frame = find_frame(rxf, cfg) # chirp matched filter
    out = decode(rxf, frame.start) # FFT, LS, track, MMSE, FEC
    ppm = estimate_clock_ppm(out.grid[:1 s]) # pilot phase-slope fit
    if abs(ppm) * n_symbols * L > 3 samples: # drift would matter
        rx2 = resample(rxf, 1 + ppm * 1e-6) # onto the TX clock
        out2 = decode(rx2, frame.start)
        out = better_of(out, out2) # by packets recovered
    return out

```

**Clock-offset estimator (sync.py):**

```

P = grid[1:, pilots] / pilot_values # [symbol, pilot]
Z = P[:, 1:] * np.conj(P[:, :-1]) # neighbouring-pilot products
for each pair i:
    rate[i] = polyfit(n, unwrap(angle(Z[:, i])), 1).slope / dk[i]
eps = weighted_mean(rate, |Z|) / (2*pi*L/N)

```

**Soft-decision Viterbi (conv.py), vectorised over the 64 trellis states:**

```

for t in range(n_steps):
    bm = BRANCH @ soft[t] # correlation metric, [state, pred]
    cand = pm[PRED] + bm
    x = argmax(cand, axis=1) # survivor choice per state
    decisions[t] = x
    pm = cand[states, x]
# trace back from state 0 (6 tail bits force the trellis there)

```

## 5.4 User Interface

Results are published as an interactive dashboard on GitHub Pages (URL in the Annexure). The page is built from `dashboard/data.json`, which is exported directly from the experiment

scripts, so no number on it is typed by hand. It shows the project status, the headline metrics, the complete signal chain (Figure 5.2), every BER experiment with hover tooltips and data tables, the real hardware measurements, and the error-correction and clock-drift results. It works in light and dark themes and on phones. A separate live demo (`live_server.py`) lets a user type a message in the browser, hear it played, and watch it decoded from the microphone, including from a second device on the same Wi-Fi.

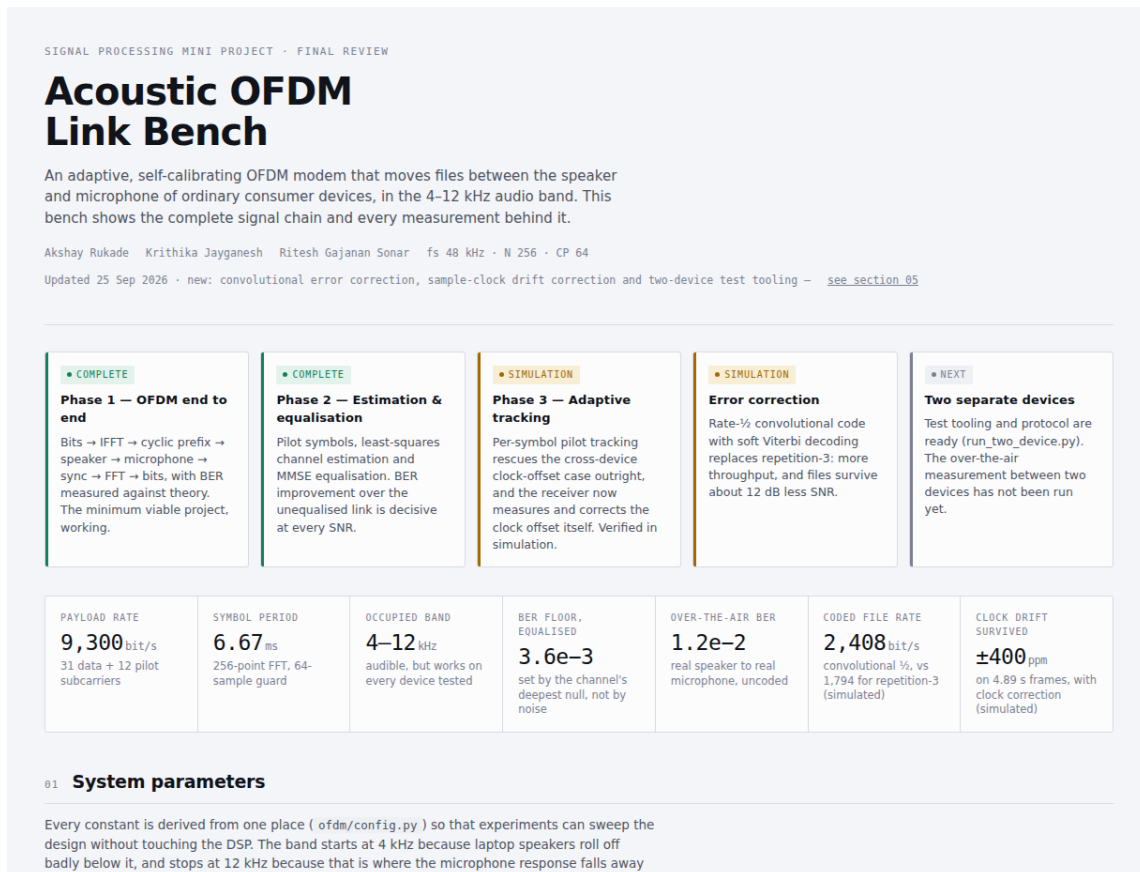


Figure 5.1: Dashboard: project status and headline metrics

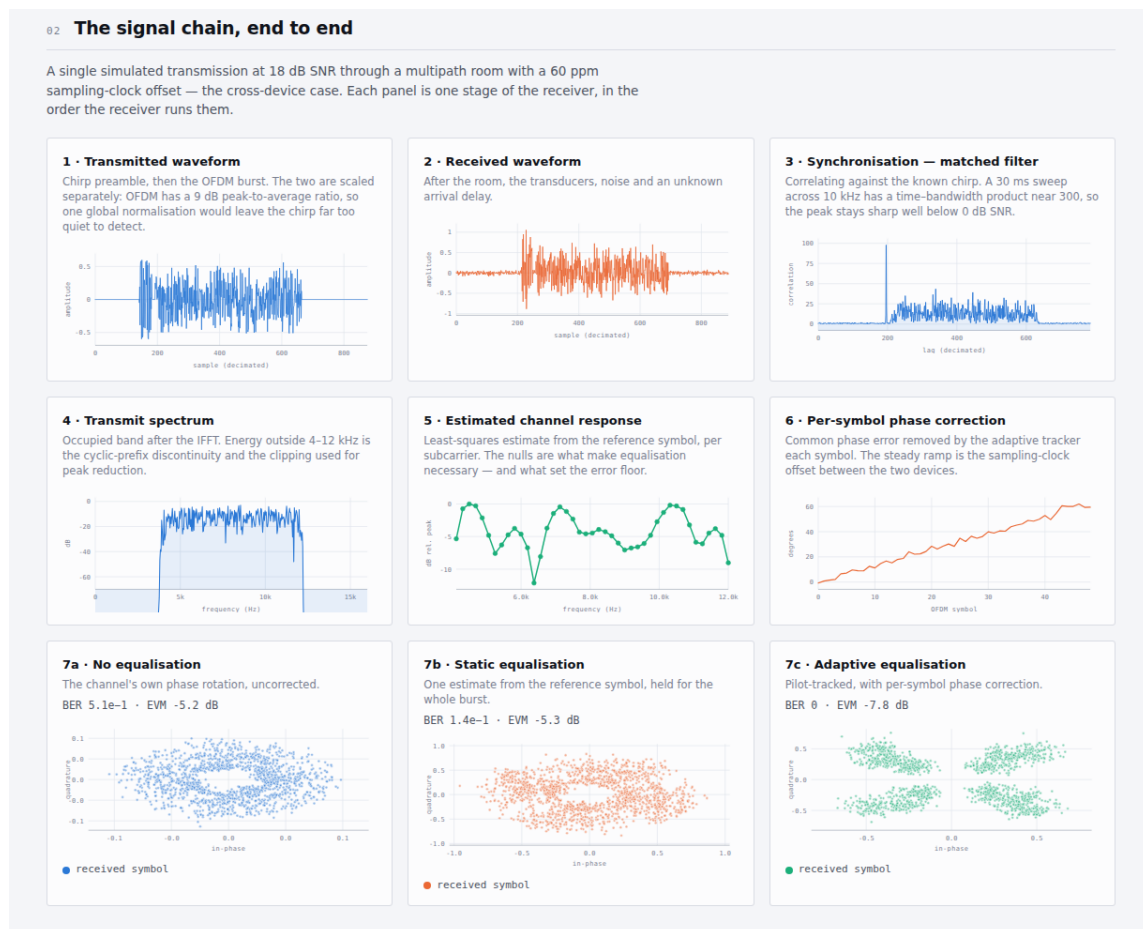


Figure 5.2: Dashboard: the signal chain end to end, one stage per panel

## Chapter 6

# Problems Faced and Solutions

### 6.1 Problem 1: The synchroniser could not find the chirp

**Issue:** The recovered BER was 0.49 with no obvious cause. The matched filter was locking onto random correlation inside the payload.

**Solution:** The whole frame had been normalised by its global peak. OFDM has a crest factor of about 9 dB and the chirp about 3 dB, so the chirp came out 6 dB too quiet. Normalising the preamble and the payload separately raised detection confidence from  $8.6\times$  to  $121\times$  the noise floor. Timing became sample-exact down to 0 dB SNR.

### 6.2 Problem 2: Pilot interpolation aliased

**Issue:** Channel estimates were nonsense even at high SNR.

**Solution:** Starting the FFT window  $D$  samples early rotates the apparent channel by  $2\pi D/N$  per subcarrier. Pilot interpolation only works while the rotation between adjacent pilots stays under  $\pi$ , i.e.  $D < N/(2 \cdot \text{pilot spacing}) = 32$ . The first backoff was exactly 32; the measured rotation was  $184^\circ$  between pilots. Reducing the backoff to 8 samples fixed it. The synchroniser's timing choice turned out to set a hard constraint on the channel estimator's pilot spacing.

### 6.3 Problem 3: Adaptive tracking made the link worse

**Issue:** Replacing the channel estimate with the pilot-interpolated one each symbol raised BER from  $3 \times 10^{-4}$  to  $10^{-1}$ . It did not improve with SNR, so it was systematic.

**Solution:** That update discards a 43-subcarrier reference estimate for 12 pilots interpolated across spectral nulls. Instead we track a smooth multiplicative correction fitted to a two-parameter physical model (gain and phase slope), leaving the null structure from the reference symbol intact. Piecewise interpolation of the correction was tried first and left a  $10^{-2}$  floor; the parametric fit removed it.

### 6.4 Problem 4: Real hardware failures that looked like DSP bugs

**Issue:** Over the air the receiver locked onto the wrong place, or decoded nothing.

**Solution:** Four separate faults. (i) The ALSA capture device emits a full-scale 10 ms click at start-up, 30 dB above the signal; the first 250 ms are now blanked. (ii) A similar transient at capture stop; a tail guard was added. (iii) The noise-floor estimate collapsed once the search

window narrowed; a 20th-percentile estimate replaced the median. (iv) The microphone gain was 34 dB down, and on one occasion the speaker was muted. Capture gain is now calibrated automatically before every transmission. A repeated-symbol diagnostic separated noise, distortion and receiver bugs, and showed the recording was clipping.

## 6.5 Problem 5: A third of the band was unusable

**Issue:** Over the air, the uncoded BER was stuck at 0.31 and no equaliser tuning moved it.

**Solution:** Measuring the real speaker + microphone + room response with the chirp showed 43 dB of variation across the band, with nulls 35–43 dB deep between 6.5 and 9 kHz. No equaliser can recover a subcarrier in such a null. Stage-1 self-calibration measures the response first and transmits only on subcarriers within a backoff of the strongest. With 8 dB backoff the BER fell to  $8.3 \times 10^{-3}$ .

## 6.6 Problem 6: Repetition coding had an error floor

**Issue:** One bit error destroys a 72-byte packet, so uncoded frames failed even in simulation. Repetition-3 helped but its decoded BER stopped at  $5 \times 10^{-3}$  at every SNR, and it spent two-thirds of the airtime on redundancy.

**Solution:** Block-tiled copies of a bit can land on the same faded subcarrier, so all three copies fail together. We replaced it with an interleaved rate- $\frac{1}{2}$  convolutional code decoded by soft-decision Viterbi (Section 7.2).

## 6.7 Problem 7: Long frames broke under clock drift

**Issue:** Frames longer than about 2.5 s failed at clock offsets of 200 ppm or more, even though adaptive tracking handles the phase rotation. Failures were asymmetric: +200 ppm failed while –200 ppm survived.

**Solution:** The FFT window slides  $\varepsilon L$  samples per symbol. It starts 8 samples inside the 64-sample prefix, so it has 56 samples of margin in one direction but only 8 in the other before symbols overlap. We added a clock-offset estimator and a resample-and-redecode pass. The first estimator differenced adjacent symbols and was off by tens of ppm (it read +80 ppm as –34). Fitting a line over the whole frame cut the error to a few ppm.

## Chapter 7

# Results and Discussion

### 7.1 Experimental Setup

- **Simulation.** A known channel lets BER be measured exactly: sparse multipath (direct path and three reflections within 1 ms), a band-pass FIR for the transducers, additive white noise at a set SNR, and sampling-clock offset by resampling. Each SNR point in a sweep sees the same set of rooms. BER experiments use  $16 \text{ trials} \times 6\,200 \text{ bits per point}$  (about 99 000 bits).
- **Hardware.** A laptop speaker to the same laptop’s microphone via ALSA at 48 kHz, carrying a 293-byte text file.
- **Two devices.** [\[Transmitter and receiver models, room, distance, volume\]](#).
- **Metrics.** Bit error rate (BER), packet recovery rate (PRR, share of 72-byte packets passing CRC), error-vector magnitude (EVM), exact-file recovery, and clock-offset estimation error.

### 7.2 Quantitative Results

| SNR                  | Measured BER            | QPSK theory           |
|----------------------|-------------------------|-----------------------|
| 0 dB                 | $7.32 \times 10^{-2}$   | $7.86 \times 10^{-2}$ |
| 2 dB                 | $3.56 \times 10^{-2}$   | $3.75 \times 10^{-2}$ |
| 4 dB                 | $1.20 \times 10^{-2}$   | $1.25 \times 10^{-2}$ |
| 6 dB                 | $3.30 \times 10^{-3}$   | $2.39 \times 10^{-3}$ |
| 8 dB                 | $4.03 \times 10^{-4}$   | $1.91 \times 10^{-4}$ |
| 10 dB                | $3.02 \times 10^{-5}$   | $3.87 \times 10^{-6}$ |
| $\geq 12 \text{ dB}$ | 0 errors in 99 200 bits | —                     |

Table 7.1: Phase 1: AWGN BER against coherent-QPSK theory

| SNR   | No equalisation       | Static                | Adaptive              |
|-------|-----------------------|-----------------------|-----------------------|
| 0 dB  | $4.73 \times 10^{-1}$ | $1.21 \times 10^{-1}$ | $1.29 \times 10^{-1}$ |
| 6 dB  | $4.69 \times 10^{-1}$ | $3.80 \times 10^{-2}$ | $3.94 \times 10^{-2}$ |
| 12 dB | $4.67 \times 10^{-1}$ | $1.21 \times 10^{-2}$ | $1.28 \times 10^{-2}$ |
| 18 dB | $4.68 \times 10^{-1}$ | $4.58 \times 10^{-3}$ | $4.83 \times 10^{-3}$ |
| 30 dB | $4.68 \times 10^{-1}$ | $3.63 \times 10^{-3}$ | $3.62 \times 10^{-3}$ |

Table 7.2: Phase 2: BER over the multipath channel by equaliser mode

| Clock offset   | Static calibration                      | Adaptive tracking                       |
|----------------|-----------------------------------------|-----------------------------------------|
| 0 ppm          | $4.47 \times 10^{-3}$                   | $4.54 \times 10^{-3}$                   |
| 25 ppm         | $5.40 \times 10^{-2}$                   | $4.02 \times 10^{-3}$                   |
| 50 ppm         | $2.20 \times 10^{-1}$                   | $3.68 \times 10^{-3}$                   |
| <b>100 ppm</b> | <b><math>5.14 \times 10^{-1}</math></b> | <b><math>3.79 \times 10^{-3}</math></b> |
| 200 ppm        | $5.14 \times 10^{-1}$                   | $4.63 \times 10^{-3}$                   |
| 400 ppm        | $5.00 \times 10^{-1}$                   | $1.05 \times 10^{-2}$                   |
| 800 ppm        | $5.02 \times 10^{-1}$                   | $1.10 \times 10^{-1}$                   |

Table 7.3: Phase 3: BER under sampling-clock offset

| Clipping limit        | PAPR          | Avg power      | BER @ 8 dB                             | BER @ 14 dB          |
|-----------------------|---------------|----------------|----------------------------------------|----------------------|
| none                  | 11.3 dB       | +0.0 dB        | $1.9 \times 10^{-4}$                   | 0                    |
| 8 dB above RMS        | 9.2 dB        | +2.1 dB        | $1.3 \times 10^{-4}$                   | 0                    |
| <b>6 dB above RMS</b> | <b>8.0 dB</b> | <b>+3.3 dB</b> | <b><math>3.2 \times 10^{-4}</math></b> | <b>0</b>             |
| 4 dB above RMS        | 7.1 dB        | +4.2 dB        | $2.6 \times 10^{-3}$                   | $5.4 \times 10^{-5}$ |

Table 7.4: Peak-to-average power reduction trade-off

| Configuration                     | BER                                                       | Packets  |
|-----------------------------------|-----------------------------------------------------------|----------|
| Full band, no calibration         | $3.1 \times 10^{-1}$                                      | 0/5      |
| Stage 1, 12 dB backoff            | $8.7 \times 10^{-2}$                                      | 0/5      |
| Stage 1, 8 dB backoff             | $8.3 \times 10^{-3}$                                      | 1/5      |
| + repetition-3                    | $1.8 \times 10^{-2} \rightarrow 1.9 \times 10^{-3}$ coded | 3/5      |
| + repetition-5                    | $5.2 \times 10^{-2} \rightarrow 6.4 \times 10^{-4}$ coded | 4/5      |
| Best run (repetition-3, 8 dB)     | $1.2 \times 10^{-2} \rightarrow 3.2 \times 10^{-4}$ coded | 4/5      |
| <b>[convolutional code, 8 dB]</b> | <b>□</b>                                                  | <b>□</b> |

Table 7.5: Over the air on real hardware (one laptop, speaker to microphone)

| Scheme                                     | Air time      | bit/s        | BER, 0 dB                              | BER, 2 dB                              | BER, 6 dB            | All intact       |
|--------------------------------------------|---------------|--------------|----------------------------------------|----------------------------------------|----------------------|------------------|
| Uncoded                                    | 0.63 s        | 3 701        | $9.0 \times 10^{-2}$                   | $5.9 \times 10^{-2}$                   | $2.4 \times 10^{-2}$ | never            |
| Repetition-3                               | 1.31 s        | 1 794        | $3.8 \times 10^{-2}$                   | $2.0 \times 10^{-2}$                   | $7.9 \times 10^{-3}$ | from 18 dB       |
| <b>Conv. rate <math>\frac{1}{2}</math></b> | <b>0.97 s</b> | <b>2 408</b> | <b><math>3.4 \times 10^{-3}</math></b> | <b><math>1.6 \times 10^{-4}</math></b> | <b>0</b>             | <b>from 4 dB</b> |

Table 7.6: Forward error correction: 293-byte file, simulated room with 80 ppm clock offset, 8 paired trials per point

| Frame               | -400  | -200  | -100  | 0     | +100  | +200  | +400 ppm |
|---------------------|-------|-------|-------|-------|-------|-------|----------|
| 2.67 s, uncorrected | 33 %  | 100 % | 100 % | 100 % | 100 % | 67 %  | 0 %      |
| 2.67 s, corrected   | 100 % | 100 % | 100 % | 100 % | 100 % | 100 % | 100 %    |
| 4.89 s, uncorrected | 0 %   | 100 % | 100 % | 100 % | 100 % | 0 %   | 0 %      |
| 4.89 s, corrected   | 100 % | 100 % | 100 % | 100 % | 100 % | 100 % | 100 %    |

Table 7.7: Frames recovered exactly under clock offset, with and without correction (simulated, 18 dB SNR, 3 trials per cell)

| Frame length | Mean absolute error | Worst error |
|--------------|---------------------|-------------|
| 0.62 s       | 21 ppm              | 31 ppm      |
| 1.18 s       | 7 ppm               | 23 ppm      |
| 2.67 s       | 5 ppm               | 11 ppm      |
| 4.89 s       | 4 ppm               | 9 ppm       |

Table 7.8: Clock-offset estimation accuracy over offsets from -400 to +400 ppm

| Setup                            | Distance | Decoded                                                                             | Exact                                                                               | Clock offset                                                                         | EVM                                                                                   | Raw BER                                                                               |
|----------------------------------|----------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| A $\rightarrow$ B, repetition-3  | 0.5 m    |  |  |  |  |  |
| A $\rightarrow$ B, repetition-3  | 1 m      |  |  |  |  |  |
| A $\rightarrow$ B, repetition-3  | 2 m      |  |  |  |  |  |
| A $\rightarrow$ B, repetition-3  | 3 m      |  |  |  |  |  |
| A $\rightarrow$ B, convolutional | 1 m      |  |  |  |  |  |
| B $\rightarrow$ A, repetition-3  | 1 m      |  |  |  |  |  |

Table 7.9: Two separate devices, 5 frames per row ([from run\_two\_device.py report])

### 7.3 Qualitative Results

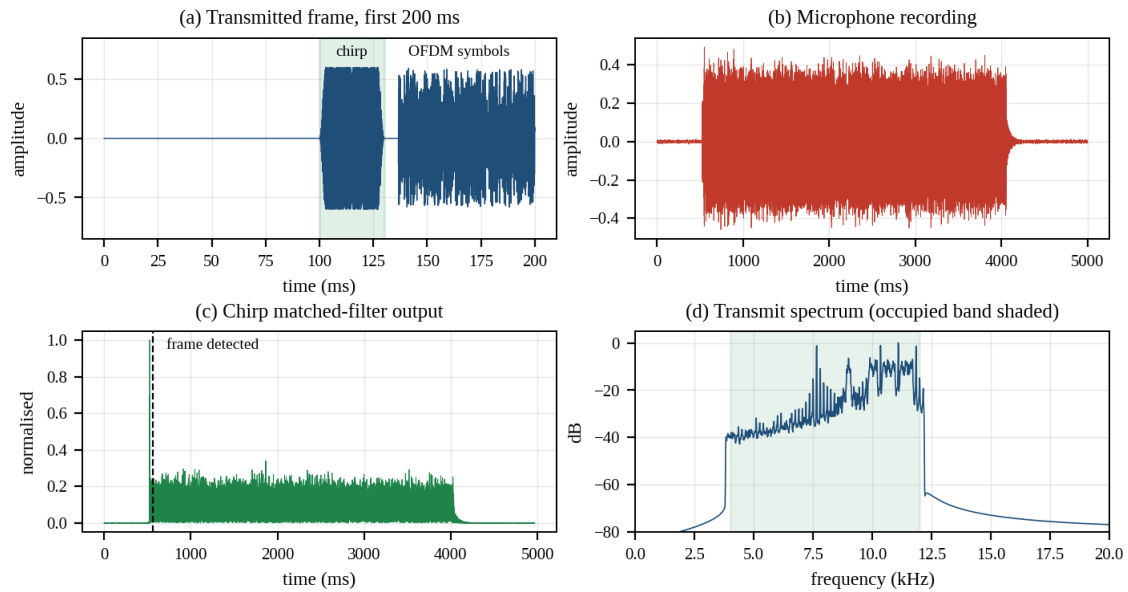


Figure 7.1: Signal chain: transmitted frame, microphone recording, chirp matched-filter output and transmit spectrum

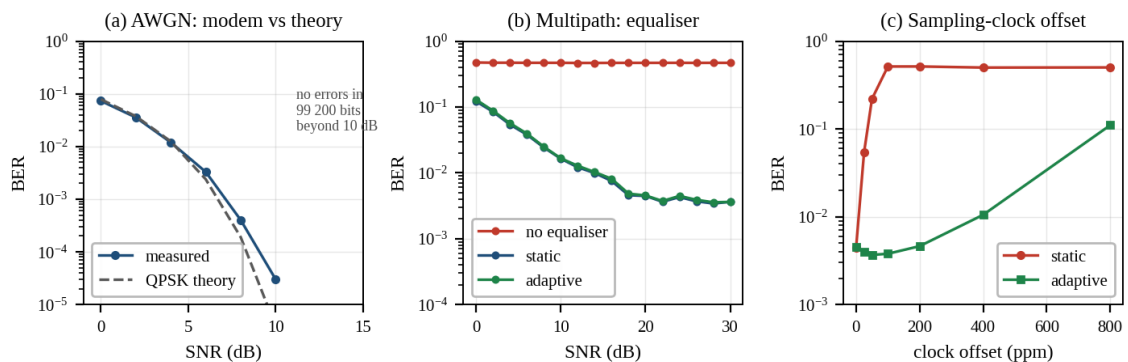


Figure 7.2: BER curves: AWGN against theory, equaliser comparison and clock offset

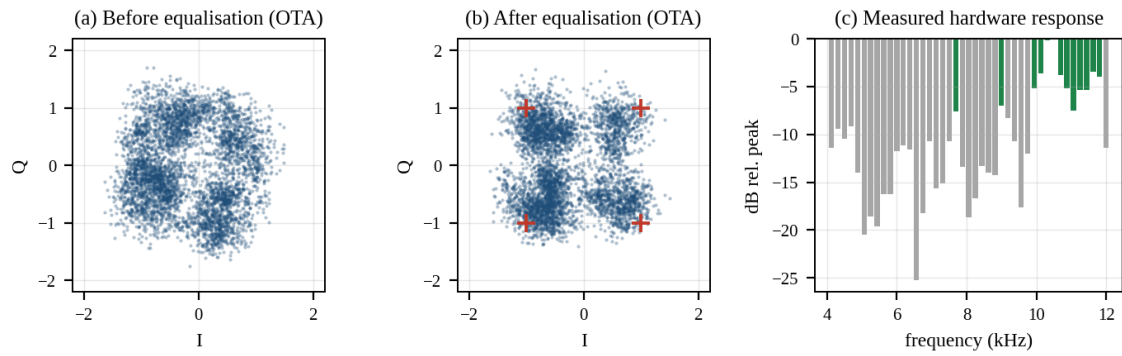


Figure 7.3: Equalised constellations

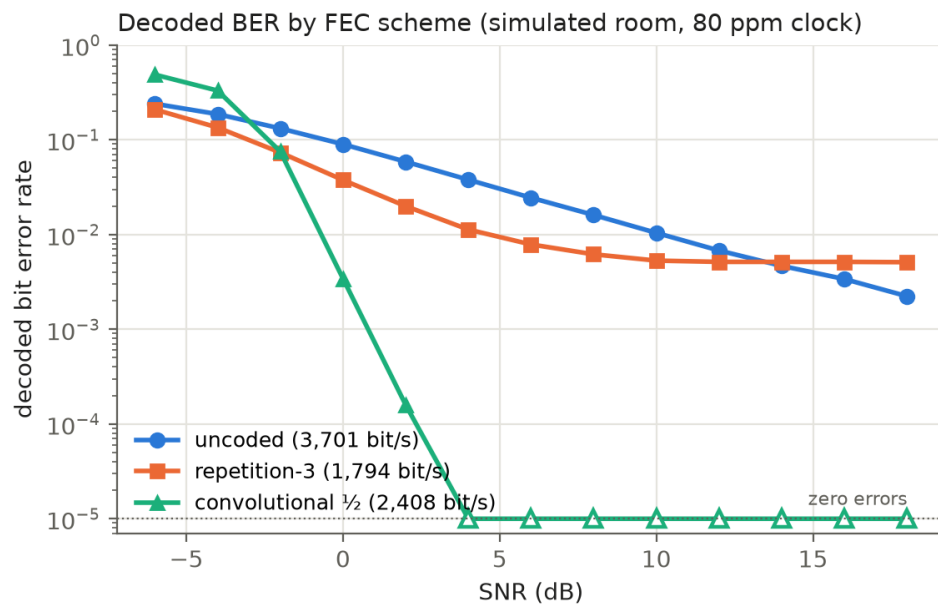


Figure 7.4: Decoded BER by FEC scheme (hollow markers: zero errors)

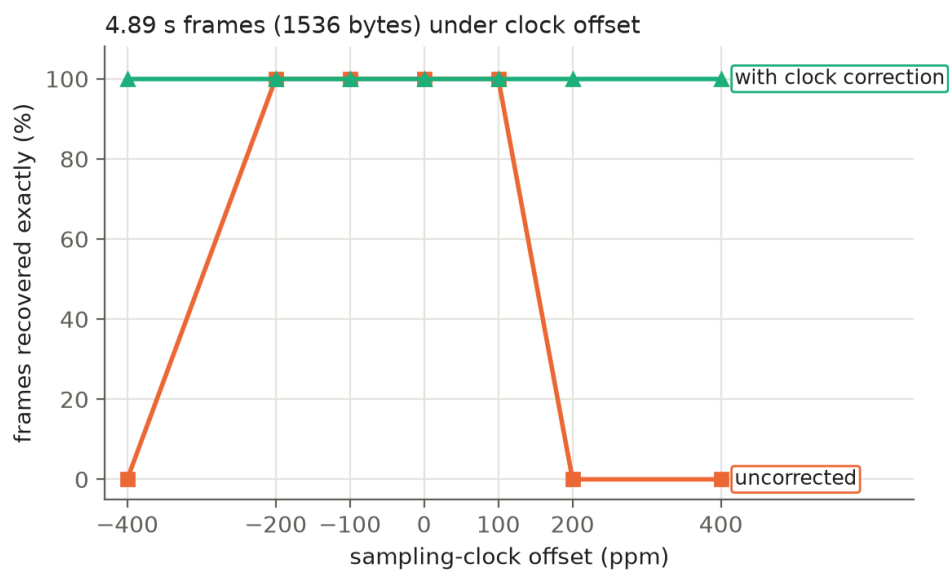


Figure 7.5: Exact frame recovery against clock offset, 4.89 s frames

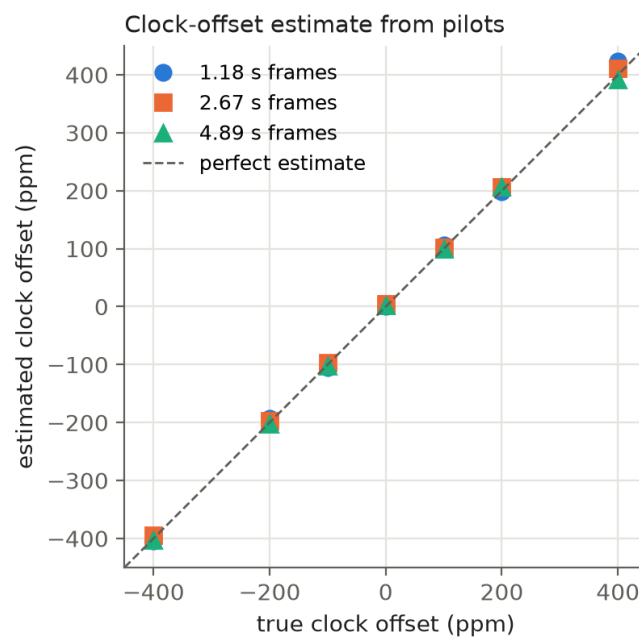


Figure 7.6: Estimated against true clock offset

## 7.4 Discussion

- **Theory is met where it should be.** On AWGN, measured BER sits on the QPSK curve from 0 to 8 dB. The small excess at high SNR is the deliberate clipping distortion from PAPR reduction, which buys 3.3 dB of transmit power.
- **Equalisation is the link, not an optimisation.** Without it, BER stays at 0.47 at every SNR: the room's frequency selectivity, not noise, destroys the constellation. The equalised floor near  $3.6 \times 10^{-3}$  is set by spectral nulls, which is why error correction mattered so much.
- **Adaptive tracking earns its place only when the channel moves.** On a fixed channel it matches static calibration exactly, so it costs nothing. Under an ordinary 100 ppm clock offset, static calibration fails completely and tracking holds the link: a  $135\times$  improvement.
- **Measuring the hardware mattered more than any receiver refinement.** Stage-1 self-calibration gave a  $37\times$  BER improvement over the air; no equaliser change came close.
- **A standard code beats an ad-hoc one.** The convolutional code is 34% faster than repetition-3 and needs about 14 dB less SNR to deliver every file intact. Soft decisions and interleaving are essential to that.
- **Two-device result.** [Measured clock offset between the devices; is it stable, and does it flip sign when roles swap? How far does each code reach? Does it confirm the simulated clock-offset result?]
- **Limitations of the evidence.** The convolutional-code and drift-correction results are from simulation. The single-laptop hardware tests share one clock. Run-to-run variation over the air is significant, because the acoustic path changes with room noise and device placement.

# Chapter 8

## Conclusions

### 8.1 Summary of Contributions

We built a complete, from-scratch OFDM acoustic modem and measured every stage of it. The measurements show that equalisation is essential over a real acoustic channel, and that one-shot calibration fails at ordinary cross-device clock offsets where adaptive tracking works. Measuring the hardware response first was the single largest improvement on real hardware. An interleaved convolutional code with soft-decision decoding, plus pilot-based clock-offset correction, make the link both faster and far more robust in simulation. [One sentence on the two-device result.]

### 8.2 Limitations

- The strongest results (convolutional code, drift correction) are verified in simulation; over-the-air confirmation is [complete / pending].
- Every subcarrier uses QPSK, so weak subcarriers are discarded rather than loaded with fewer bits; the uncoded equalised error floor remains.
- Throughput is about 2.4 kbit/s of file data: suitable for text and small files only.
- The 4–12 kHz band is audible, and performance depends strongly on volume, distance and room noise.
- Calibration assumes both ends can be configured together; there is no feedback channel.

### 8.3 Future Work

- **Bit-loading:** BPSK on weak subcarriers and 16-QAM on strong ones, driven by the Stage-1 measurement, to remove the error floor and raise throughput.
- **Decision-directed tracking** between pilots for more tracking bandwidth without extra overhead.
- **A feedback handshake** so the receiver can report its chosen subcarrier set to the transmitter.
- **A mobile implementation** of the receiver, so a phone can decode as well as transmit.

## Chapter 9

# Individual Contribution

This was a three-member group project (Akshay Rukade, J Krithika and Ritish Gajanan Sonar). The work was divided by module; integration debugging and the experiment scripts were shared. My own contribution (Akshay Rukade, OFDM modulation/demodulation engine):

I owned the modem core (`ofdm/modem.py`, `ofdm/config.py`): turning a bit vector into a waveform a speaker can play, and received samples back into a resource grid.

- **Real-valued OFDM without a mixer:** designed the Hermitian-symmetric IFFT mapping (QPSK on bins 22–64 of a 256-point IFFT), so the transmitter output is already a real 4–12 kHz audio signal. Verified numerically: the residual imaginary part is  $7 \times 10^{-17}$ . This removed the up/down-conversion stage and every carrier-offset problem with it.
- **System parameter set:** chose and justified the sample rate, FFT size, cyclic prefix, band and modulation (Section 5.2), and put them in one dataclass with derived properties. This let Stage-1 calibration later restrict the modem to a subset of subcarriers with a one-line change.
- **Pilots and preamble:** a pseudo-random BPSK pilot sequence (an all-ones pattern produces a loud, clipping impulse), a fully-known reference symbol on all 43 subcarriers for the one-shot channel estimate, and the raised-cosine-tapered 30 ms synchronisation chirp.
- **Peak-to-average power reduction:** implemented iterative clipping and frequency-domain filtering and ran the trade-off sweep (Table 7.4). Chose clipping at 6 dB above RMS, which recovers 3.3 dB of average transmit power and still reaches zero errors from 14 dB SNR.
- **Debugging:** found that normalising the whole frame by one global peak left the chirp 6 dB too quiet to detect (Problem 6.1). Separate normalisation raised detection confidence from  $8.6\times$  to  $121\times$  the noise floor.
- **Verification of the module:** ideal loopback BER = 0, AWGN BER against theory, PAPR reduction preserving demodulation, and correct re-derivation of pilot and payload sets for a restricted subcarrier configuration.
- [Post-mid-term work: add what you did on the convolutional code, clock-offset estimation and correction, the two-device test and measurements, the test suite or the dashboard, or delete this item.]
- **Documentation:** individual mid-term report (Member 1: modem engine); [sections of this report you wrote].

# Chapter 10

## Annexure

### Important: Access Permissions

All linked files must have “Anyone with the link” access. If files are inaccessible during evaluation, no marks will be awarded for those components.

### 10.1 Video Demonstration

**Google Drive Link:** [PASTE YOUR VIDEO DEMO LINK HERE]

The video should clearly demonstrate:

- The working application/system end-to-end
- Key features and outputs
- Any hardware involved (if applicable)
- Brief walkthrough of the UI

### 10.2 Source Code

**Google Drive Link (ZIP file):** [PASTE YOUR CODE ZIP LINK HERE]

**GitHub repository:** <https://github.com/RukadeAkshay01/acoustic-ofdm>

The ZIP file should contain:

- All source code files (organized in folders)
- README.md with setup and run instructions
- requirements.txt or equivalent dependency list
- Sample input data (if applicable)

### 10.3 Live Deployment (if applicable)

**App URL:** <https://rukadeakshay01.github.io/acoustic-ofdm/>

# References

1. A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007.
3. J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008.
4. J. Armstrong, “Peak-to-average power reduction for OFDM by repeated clipping and frequency domain filtering,” *Electronics Letters*, vol. 38, no. 5, pp. 246–247, 2002.
5. M. Speth, S. A. Fechtel, G. Fock and H. Meyr, “Optimum receiver design for wireless broad-band systems using OFDM—Part I,” *IEEE Trans. Communications*, vol. 47, no. 11, pp. 1668–1677, 1999.
6. A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Trans. Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.
7. IEEE Std 802.11a-1999, *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications: High-speed Physical Layer in the 5 GHz Band*.